

**Name:** Qadeer durrani

**Roll no :**su91-bsram-f25-040

**Dept :** Robotics and ai

**Oops**

**(labs 6)**

## Task 1

### Robot Sensor Logger

Simulate a robot collecting sensor data.

Write a program that:

❑ Creates a file named `sensor_log.txt`

❑ Stores 3 sensor readings, each with: Sensor ID, Temperature (float), Distance in cm (float),

Timestamp (string)

❑ Reads back and displays all readings on screen

**Real-world context:** Every industrial robot and autonomous vehicle logs sensor data continuously to files

for safety analysis, debugging, and AI model training.

The image shows a C++ program in a code editor and its execution output in a terminal window. The program, named `lab6complete.cpp`, defines a `SensorData` struct with fields for `sensorID`, `temperature`, `distance`, and `timestamp`. It then implements a `main` function that creates an output file `sensor_log.txt`, prompts the user to enter details for 3 sensor readings, and stores each reading in an array. Finally, it reads back all stored readings and displays them on the screen.

```
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4 using namespace std;
5 struct SensorData
6 {
7     string sensorID;
8     float temperature;
9     float distance;
10    string timestamp;
11};
12 int main()
13 {
14    SensorData sensor[3];
15    ofstream outFile("sensor_log.txt");
16    if (!outFile)
17    {
18        cout << "Error creating file!" << endl;
19        return 1;
20    }
21    cout << "Enter details for 3 sensor readings:\n";
22    for (int i = 0; i < 3; i++)
23    {
24        cout << "\nReading " << i + 1 << endl;
25        cout << "Sensor ID: ";
26        cin >> sensor[i].sensorID;
27        cout << "Temperature: ";
28        cin >> sensor[i].temperature;
29        cout << "Distance (cm): ";
30        cin >> sensor[i].distance;
31        cout << "Timestamp: ";
32        cin >> sensor[i].timestamp;
33        outFile << sensor[i].sensorID << " "
34                << sensor[i].temperature << " "
35                << sensor[i].distance << " "
36                << sensor[i].timestamp << endl;
```

The terminal window shows the program's execution. It prompts the user to "Enter details for 3 sensor readings:" and displays the input for each reading. Reading 1 has Sensor ID: 45, Temperature: 23, Distance (cm): 120, and Timestamp: 458. Reading 2 has Sensor ID: 154, Temperature: 22, Distance (cm): 450, and Timestamp: 850.

```
Enter details for 3 sensor readings:

Reading 1
Sensor ID: 45
Temperature: 23
Distance (cm): 120
Timestamp: 458

Reading 2
Sensor ID: 154
Temperature: 22
Distance (cm): 450
Timestamp: 850
```

## Task 2: Student Database with File Backup

Build a mini student record system.

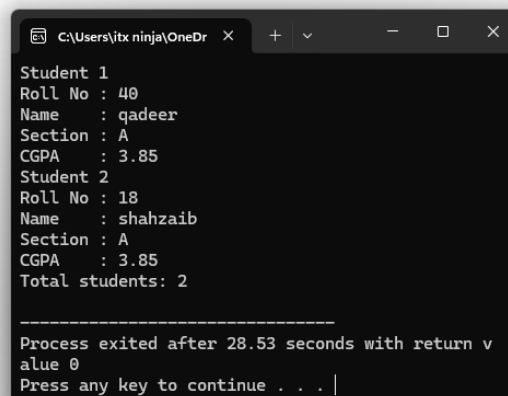
Write a program that:

- ❑ Stores data for 2 students: Roll No, Name, Section, CGPA
- ❑ Writes all data to student.txt
- ❑ Reads back and prints each student's record with a formatted header
- ❑ Appends a summary line: Total students: 2

Real-world context: Database backup to flat files is a core DevOps practice. Tools like MySQL and

PostgreSQL export data to .csv / .txt for portability

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
struct Student
{
    int rollNo;
    string name;
    string section;
    float cgpa;
};
int main()
{
    Student s[2];
    ofstream outFile("student.txt");
    cout << "Enter details of 2 students:";
    for (int i = 0; i < 2; i++)
    {
        cout << "Student " << i + 1 << endl;
        cout << "Roll No: ";
        cin >> s[i].rollNo;
        cout << "Name: ";
        cin >> s[i].name;
        cout << "Section: ";
        cin >> s[i].section;
        cout << "CGPA: ";
        cin >> s[i].cgpa;
        outFile << s[i].rollNo << " "
                << s[i].name << " "
                << s[i].section << " "
                << s[i].cgpa << endl;
    }
    outFile.close();
    ofstream appendFile("student.txt", ios::app);
    appendFile << "Total students: 2" << endl;
```



```
C:\Users\itx ninja\OneDr
Student 1
Roll No : 40
Name : qadeer
Section : A
CGPA : 3.85
Student 2
Roll No : 18
Name : shahzaib
Section : A
CGPA : 3.85
Total students: 2

-----
Process exited after 28.53 seconds with return v
alue 0
Press any key to continue . . . |
```

### Task 3: Bank Account File System

Create a banking record system. Store records for 5 account holders:

- ❑ Fields: Name, Account Number, 30-day deposits (array), Current Balance
- ❑ Write all records to details.txt
- ❑ Read back and display only accounts with balance > 50,000
- ❑ Use dynamic memory (new / delete) for the deposit array

Real-world context: Financial systems write daily transaction logs to files for audit trails a legal requirement

in the banking industry.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
struct Account
{
    string name;
    long long accountNumber;
    float *deposits;
    float currentBalance;
};
int main()
{
    const int SIZE = 5;
    Account acc[SIZE];
    ofstream outFile("details.txt");
    if (!outFile)
    {
        cout << "File could not be created." << endl;
        return 1;
    }
    cout << "Enter details of 5 account holders:";
```

```
Enter details of 5 account holders:Account Holder 1
Name: Qadeer
Account Number: 14101101774
Enter 30-day deposits:300000
15000000000000
1880446
0245460
18887446
154546
112545745
```

## Home task

### Task 1: Autonomous Robot Path Logger (CSV Format)

Write a C++ program that simulates a robot recording its movement path.

**Requirements:**

- ❑ Log at least 10 waypoints to path\_log.csv in format: waypoint\_id, x, y, z, timestamp
- ❑ Read the CSV back and calculate: total distance traveled (Euclidean), average speed
- ❑ Detect and flag any waypoints that are outside a safe zone (e.g.  $x > 100$ )
- ❑ Append a summary section at the end of the file

**Industry connection:**

- ❑ ROS2 (Robot Operating System) bags data in structured files.
- ❑ Self-driving cars log GPS waypoints in CSV for route replay and fault analysis.
- ❑ Tools like Pandas (Python) and C++ file I/O process these logs in production.

```
#include <iostream>
#include <fstream>
#include <cmath>
#include <string>
using namespace std;
struct Waypoint
{
    int id;
    float x, y, z;
    int timestamp;
};
int main()
{
    const int SIZE = 10;
    Waypoint wp[SIZE];
    ofstream outFile("path_log.csv");
    if (!outFile)
    {
        cout << "File Error!" << endl;
        return 1;
    }
    outFile << "waypoint_id,x,y,z,timestamp";
    for (int i = 0; i < SIZE; i++)
    {
        wp[i].id = i + 1;
        cout << "Waypoint " << i + 1 << endl;
```

```
Timestamp: 57
Waypoint 10
X: 57
Y: 75
Z: 75
Timestamp: 57
===== PATH REPORT =====Total Distance Travelled: 115730
Average Speed: 0
Unsafe Waypoints (x > 100)No unsafe waypoints found.

Process exited after 30.66 seconds with return value 0
Press any key to continue . . .
```

## Task 2: Neural Network Weight Storage (Binary File)

Simulate saving and loading a small neural network configuration using binary files.

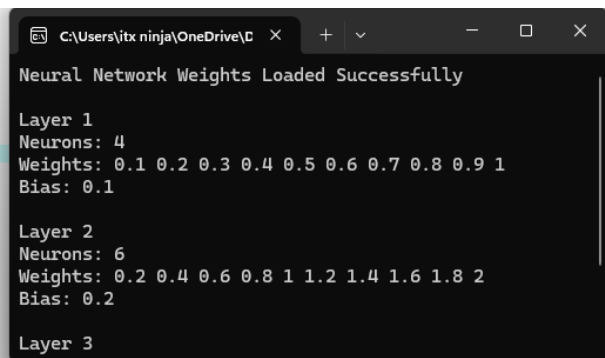
Requirements:

- ❑ Define a struct: `Layer { int neurons; float weights[10]; float bias; }`
- ❑ Dynamically allocate an array of 3 layers using `new`
- ❑ Fill with hardcoded values representing a simple 3-layer network
- ❑ Write the entire array to `model_weights.bin` using binary write
- ❑ In the same program, read back and verify: print all weights and biases
- ❑ Free all memory with `delete[]`

Industry connection:

- ❑ PyTorch and TensorFlow save model weights as binary files
- ❑ (.pt, .h5, .bin). Embedded AI on robots (NVIDIA Jetson, RPI) loads these at startup.
- ❑ Understanding binary I/O is foundational for deploying AI models on hardware.

```
#include <iostream>
#include <fstream>
using namespace std;
struct Layer
{
    int neurons;
    float weights[10];
    float bias;
};
int main()
{
    const int SIZE = 3;
    Layer *network = new Layer[SIZE];
    network[0].neurons = 4;
    network[0].bias = 0.1f;
    for (int i = 0; i < 10; i++)
    {
        network[0].weights[i] = 0.1f * (i + 1);
    }
    network[1].neurons = 6;
    network[1].bias = 0.2f;
    for (int i = 0; i < 10; i++)
    {
        network[1].weights[i] = 0.2f * (i + 1);
    }
    network[2].neurons = 2;
    network[2].bias = 0.3f;
    for (int i = 0; i < 10; i++)
    {
        network[2].weights[i] = 0.3f * (i + 1);
    }
}
```



```
Neural Network Weights Loaded Successfully

Layer 1
Neurons: 4
Weights: 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1
Bias: 0.1

Layer 2
Neurons: 6
Weights: 0.2 0.4 0.6 0.8 1 1.2 1.4 1.6 1.8 2
Bias: 0.2

Layer 3
```

## Task 3: Multi-Robot Fleet Log Merger

Simulate a fleet of 3 robots each writing to its own log file, then merge them.

Requirements:

- ❑ Create 3 files: `robot1_log.txt`, `robot2_log.txt`, `robot3_log.txt`
- ❑ Each file contains: Robot ID, 5 sensor readings (temperature, battery %)

❓ Write a merge function that reads all 3 files and writes fleet\_log.txt

❓ fleet\_log.txt should contain all readings sorted by Robot ID

❓ Flag any robot with battery < 20% as CRITICAL in the merged log

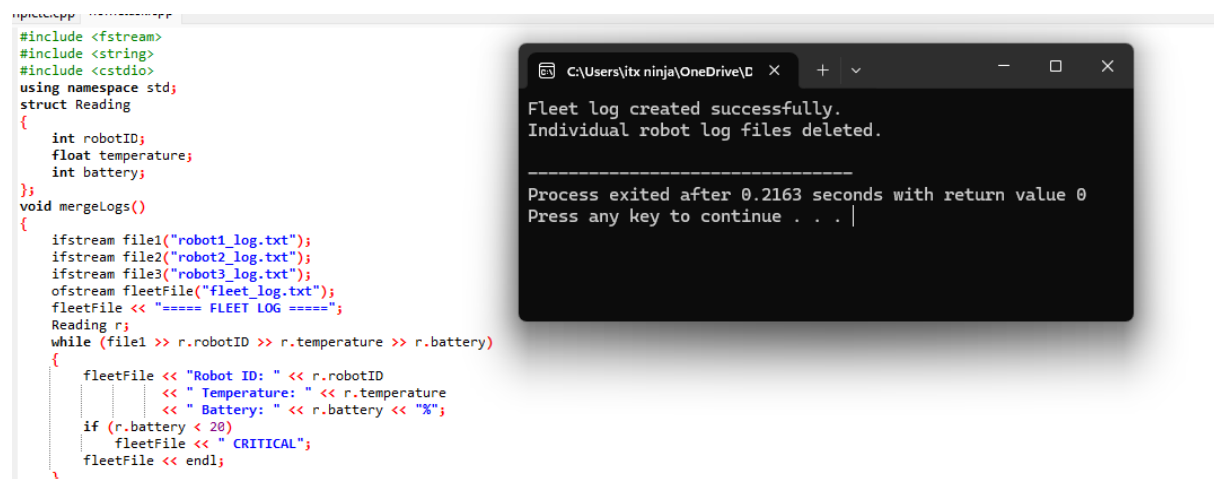
❓ Delete the 3 individual log files after merging using remove()

Industry connection:

In multi-robot systems (Amazon warehouses, drone swarms), a central server collects and merges logs from

all agents. This is standard practice in fleet management software and is directly applicable to ROS2 multi-

agent setups.



The image shows a C++ source code file on the left and its terminal output on the right. The code defines a 'Reading' struct with robotID, temperature, and battery fields. It implements a 'mergeLogs()' function that opens three input files (robot1\_log.txt, robot2\_log.txt, robot3\_log.txt), reads their contents into a vector, sorts them by robotID, and writes the sorted data to a new file 'fleet\_log.txt'. It also checks for robots with battery levels below 20% and flags them as 'CRITICAL'. Finally, it deletes the three input files. The terminal output on the right shows the successful execution of the program, displaying the message 'Fleet log created successfully. Individual robot log files deleted.' and 'Process exited after 0.2163 seconds with return value 0'.

```
#include <fstream>
#include <string>
#include <vector>
#include <algorithm>
#include <cstdio>
using namespace std;
struct Reading
{
    int robotID;
    float temperature;
    int battery;
};
void mergeLogs()
{
    ifstream file1("robot1_log.txt");
    ifstream file2("robot2_log.txt");
    ifstream file3("robot3_log.txt");
    ofstream fleetFile("fleet_log.txt");
    fleetFile << "==== FLEET LOG ====";
    Reading r;
    while (file1 >> r.robotID >> r.temperature >> r.battery)
    {
        fleetFile << "Robot ID: " << r.robotID
                  << " Temperature: " << r.temperature
                  << " Battery: " << r.battery << "%";
        if (r.battery < 20)
            fleetFile << " CRITICAL";
        fleetFile << endl;
    }
}
```

```
C:\Users\itx ninja\OneDrive\...
Fleet log created successfully.
Individual robot log files deleted.

-----
Process exited after 0.2163 seconds with return value 0
Press any key to continue . . .
```